

HAVEN

Rapport Technique Officiel — V0

Idle visuel : un petit monde miniature qui se restaure. Projet indépendant, développé pendant l'indisponibilité du poste principal, via une session Claude Code connectée en Wi-Fi. Aucun code ni asset partagé avec FRONTIÈRES, RUPTURE ou MURPHY.

Dépôt : **scuillerjack-oss/haven-v0** · Branche : **main**

1. Contexte et périmètre de la V0

Cette V0 répond au cahier des charges « PROJET 3 — HAVEN — V0 » : obtenir rapidement une version testable sur téléphone via Wi-Fi, sans dépendance à un environnement cloud spécifique, sans AAB ni Google Play, sans backend ni monétisation réelle.

Contenu livré

- Une scène unique, en portrait, occupant l'essentiel de l'écran.
- 4 systèmes de production : Source, Sol, Flore, Faune — débloqués progressivement.
- 15 améliorations nommées (aucune ne se limite à un simple « +10 % »).
- 7 paliers visuels cumulatifs (terre stérile → havre vivant).
- Sauvegarde locale automatique + progression hors-ligne plafonnée et honnêtement expliquée.
- Audio ambiant instrumental (WebAudio pur, aucun fichier à charger), mute/volume fonctionnels.
- Onboarding contextuel (3 indices maximum).
- Instrumentation locale simple (sessions, paliers, retours) — jamais envoyée nulle part.

Explicitement hors périmètre (comme demandé)

- Plusieurs mondes, live ops, missions quotidiennes, boutique, cosmétiques lourds.
- Backend, compte, cloud save, multijoueur, LLM.
- Monétisation réelle, AAB, publication Google Play.
- Premier mini-cycle/prestige : non construit en V0, faute de décision réelle qu'il changerait à ce stade (voir recommandations V1).

2. Architecture

JavaScript vanilla (ES modules), sans framework UI, bundlé avec Vite. Le rendu de la scène est un unique SVG dont les calques se révèlent progressivement — jamais d'échange brutal d'image entre paliers.

Persistance via **localStorage**. PWA installable (manifest + service worker). Déploiement automatique sur GitHub Pages via GitHub Actions à chaque push sur **main**.

Découpage des modules

Module	Rôle
src/engine/balance.js	Toutes les constantes d'équilibrage (paliers, coûts, taux, plafond hors-ligne) — aucune valeur magique ailleurs.
src/engine/state.js	Forme de l'état initial d'une partie.
src/engine/simulation.js	Logique pure : production, tap, achats, calcul hors-ligne, détection de paliers traversés.
src/engine/migrations.js	Registre de migrations de sauvegarde entre versions ; rejette proprement une sauvegarde illisible ou future.
src/save.js	Chargement/sauvegarde localStorage, au-dessus des migrations.

src/ui/scene.js	Calques SVG et règles de révélation par palier.
src/ui/hud.js	En-tête, tiroir d'améliorations, formatage de l'affichage.
src/ui/onboarding.js	3 indices contextuels, jamais plus.
src/ui/welcomeBack.js	Écran de retour : durée réelle, gain réel, plafond, paliers traversés.
src/audio/audio.js	Nappe instrumentale calme générée en WebAudio pur.
src/telemetry.js	Instrumentation locale plafonnée (sessions, paliers, retours).
src/main.js	Assemblage, boucle principale, réconciliation du temps écoulé, interactions.

Principe de conception clé : une seule source de vérité pour les paliers

Un bug a été identifié et corrigé pendant le développement : un tap peut faire franchir un seuil de palier *entre deux ticks* de la boucle principale. Une détection basée sur le delta de croissance d'un seul tick ne voit jamais ce franchissement. La correction retenue est une fonction unique `syncStage()`, appelée après chaque action qui peut faire croître le monde (tap, tick, retour d'absence), qui compare le palier actuel au dernier palier affiché et ne dépend jamais de la source du gain.

3. Boucle de jeu et courbe mesurée

La courbe n'est pas une estimation à la main : elle vient d'un script de simulation (`npm run simulate`) qui rejoue des profils de jeu réalistes — sessions actives courtes, app réellement fermée entre deux (le calcul hors-ligne plafonné s'applique, exactement comme en jeu), jamais un tick continu déguisé en absence.

Léger : 2 sessions/jour de 1 min

Palier	P0	P1	P2	P3	P4	P5	P6
Temps atteint	< 1 min	< 1 min	12.0 h	12.0 h	1.0 j	1.5 j	3.0 j

P0=Terre stérile · P1=Premier filet d'eau · P2=Mousse et humus · P3=Premières pousses · P4=Broussailles · P5=Retour de la faune · P6=Havre vivant

Taps totaux : 360 · Gain hors-ligne cumulé : 2161152 · Vitalité cumulée finale : 2171659

Modéré : 3 sessions/jour de 3 min

Palier	P0	P1	P2	P3	P4	P5	P6
Temps atteint	< 1 min	< 1 min	1 min	8.0 h	16.0 h	1.0 j	1.7 j

P0=Terre stérile · P1=Premier filet d'eau · P2=Mousse et humus · P3=Premières pousses · P4=Broussailles · P5=Retour de la faune · P6=Havre vivant

Taps totaux : 900 · Gain hors-ligne cumulé : 2075497 · Vitalité cumulée finale : 2102389

Assidu : 5 sessions/jour de 5 min

Palier	P0	P1	P2	P3	P4	P5	P6
Temps atteint	< 1 min	< 1 min	< 1 min	2 min	4.8 h	9.7 h	1.2 j

P0=Terre stérile · P1=Premier filet d'eau · P2=Mousse et humus · P3=Premières pousses · P4=Broussailles · P5=Retour de la faune · P6=Havre vivant

Taps totaux : 3600 · Gain hors-ligne cumulé : 1547204 · Vitalité cumulée finale : 1620033

Onglet laissé ouvert en continu, jamais d'achat (contrôle)

Palier	P0	P1	P2	P3	P4	P5	P6
Temps atteint	< 1 min	1 min	17 min	2.2 h	1.2 j	non atteint	non atteint

P0=Terre stérile · P1=Premier filet d'eau · P2=Mousse et humus · P3=Premières pousses · P4=Broussailles ·
P5=Retour de la faune · P6=Havre vivant

Taps totaux : 0 · Gain hors-ligne cumulé : 0 · Vitalité cumulée finale : 129600 · plafonné à 10.0 j sans atteindre le dernier palier

Lecture : la première transformation arrive toujours en moins d'une minute, quel que soit le profil — c'est la promesse « la première action utile et la première transformation doivent arriver rapidement ». La transformation finale, elle, demande entre environ 1 jour (jeu assidu) et 3 jours (jeu léger, deux courtes visites par jour) — ce qui recoupe directement la fenêtre de test demandée de 48 à 72 h. Un onglet laissé ouvert en continu, sans jamais rien acheter, plafonne bien avant la fin : l'engagement (achats, retours) reste nécessaire, la simple présence passive ne suffit pas à tout débloquent.

4. Tests automatisés

Suite node:test (aucune dépendance externe), 23 tests, tous verts au moment de la livraison.

- **Production** : la source produit dès le début (automatisation précoce) ; un tick fait avancer vitality et totalGrowth ensemble.
- **Tap** : valeur immédiate correcte, compteur de taps incrémenté.
- **Achats** : refus si verrouillé, refus si trop cher, acceptation et débit exact du coût sinon, refus au niveau maximum.
- **Amélioration globale** : verrouillée puis achetable une seule fois ; le Sentier de méditation multiplie bien la valeur du tap.
- **Paliers** : détection de chaque palier traversé, y compris plusieurs d'un coup (gros saut).
- **Hors-ligne** : plafond par défaut (4h) et étendu (8h avec Rosée nocturne) ; efficacité réduite systématiquement inférieure à l'équivalent actif ; absence négative ou nulle ne rapporte jamais rien ; survie sans erreur ni valeur invalide à une absence de 48 à 72 h.
- **Sauvegarde** : état neuf sans sauvegarde existante ; sauvegarde puis chargement redonne un état équivalent (reprise après rechargement) ; sauvegarde corrompue (JSON invalide) rejetée proprement, jamais un crash ; sauvegarde d'une version future inconnue rejetée proprement ; suppression de sauvegarde fonctionnelle.
- **Migrations** : rejet d'une entrée nulle/non-objet/sans version ; acceptation d'une sauvegarde déjà à jour ; rejet propre d'une version future inconnue.

Test manuel supplémentaire (navigateur, Playwright headless) : rendu de la scène, tap visible avec le flottant « +N », ouverture du tiroir, achat réel, transition de palier avec mise à jour visuelle immédiate, écran de retour après une absence simulée de 60 h avec plafond et paliers traversés listés honnêtement — aucune erreur console dans tous les cas testés.

5. Limites connues de la V0

- **Art de la scène minimal** : formes géométriques simples (cercles, ellipses, tracés), pas d'illustration dessinée — volontairement, pour ne pas retarder la bêta sur du cosmétique lourd (hors périmètre explicite).
- **Icônes PWA en SVG** : fonctionnent sur la plupart des navigateurs mobiles modernes, mais un écran d'accueil iOS plus ancien peut préférer un PNG — à vérifier lors du premier test réel.
- **Pas de mini-cycle/prestige** : volontairement non construit, faute de décision de gameplay réelle qu'il changerait à ce stade.
- **Un seul appareil à la fois** : la sauvegarde est locale à l'appareil (aucun cloud save, comme demandé) — jouer sur deux appareils donne deux mondes indépendants.
- **Déploiement GitHub Pages** : nécessite l'activation manuelle, une seule fois, de Pages avec la source « GitHub Actions » dans les réglages du dépôt (Settings → Pages) ; le premier run de la CI a échoué au job de déploiement pour cette raison exacte (le build et les tests, eux, sont passés) — voir procédure en section 7.

- **Courbe calibrée par simulation, pas encore par un vrai joueur** : les chiffres de la section 3 viennent d'un script qui rejoue des profils de jeu ; ils n'ont pas encore été confrontés à un test humain réel sur 48-72 h — c'est justement le test que cette bêta doit permettre.

6. Temps consommé

Développé en une session continue de Claude Code (connexion Wi-Fi, sans dépendance à un environnement cloud spécifique pour le jeu lui-même). Pas de chronomètre précis à la minute, mais les phases de travail, dans l'ordre :

- Cadrage du conflit d'instructions (le dépôt de session pointait vers RUPTURE, explicitement interdit) et création d'un dépôt indépendant haven-v0.
- Moteur de jeu (balance, état, simulation, migrations) et sauvegarde locale, avec tests au fur et à mesure.
- Interface (scène SVG à calques, tiroir d'améliorations, onboarding, écran de retour, audio).
- Vérification réelle en navigateur (Playwright headless) : un bug de détection de palier (tap entre deux ticks) trouvé et corrigé à ce stade.
- Script de simulation de profils de jeu réalistes ; premier passage révélant un équilibre bien trop rapide (partie finie en 21 minutes même sans jamais toucher le jeu) ; rééquilibrage des coûts et des paliers ; nouvelle simulation confirmant une courbe cohérente avec la fenêtre 48-72 h.
- Mise en place du déploiement GitHub Pages, commit, push, rapport.

7. Procédure exacte pour ouvrir la bêta sur le téléphone

Étape unique côté GitHub (une seule fois)

Sur github.com, ouvrir le dépôt **scuillerjack-oss/haven-v0** → Settings → Pages → « Build and deployment » → Source : **GitHub Actions**. Puis relancer le workflow « Deploy to GitHub Pages » depuis l'onglet Actions (bouton « Re-run all jobs »), ou attendre le prochain push sur main.

Sur le téléphone

- Ouvrir <https://scuillerjack-oss.github.io/haven-v0/> dans le navigateur (n'importe quel Wi-Fi ou réseau mobile — aucun ordinateur requis une fois Pages activé).
- Menu du navigateur → « Ajouter à l'écran d'accueil » pour l'installer comme une app.
- Jouer normalement ; fermer l'app à volonté — la sauvegarde reste sur l'appareil.

Alternative sans attendre l'activation de Pages : `npm run dev:lan` depuis un ordinateur sur le même Wi-Fi que le téléphone, puis ouvrir l'URL locale affichée dans le terminal depuis le navigateur du téléphone.

8. Recommandations pour la V1

- Confronter la courbe à un vrai test humain sur 48-72 h et ajuster les coûts/paliers en conséquence — la simulation est un point de départ, pas une vérité définitive.

- Envisager un premier mini-cycle/prestige seulement s'il ouvre une vraie décision de gameplay (par exemple : recommencer un monde avec un bonus permanent qui change l'ordre d'achat optimal), jamais comme un simple multiplicateur.
- Remplacer les icônes PWA SVG par des PNG multi-résolutions si le test sur iOS plus ancien montre un problème d'affichage à l'écran d'accueil.
- Envisager une illustration un peu plus travaillée de la scène si le retour du test humain montre que la lisibilité des paliers intermédiaires (mousse, pousses) manque de clarté sur petit écran.
- Étudier un deuxième monde ou une variante seulement après validation du cœur de boucle sur cette V0 — pas avant (hors périmètre explicite de la V0).

Fin du rapport — HAVEN V0, dépôt scuillerjack-oss/haven-v0, branche main.